ACCENTURE

OA + Interview Mastery

Personalized Stepwise Guide for **Rishikesh Saroj**
Mechanical @ IIITDM Jabalpur • Python / FastAPI / PostgreSQL / React / ML

This guide teaches you **exactly what is on your resume — to depth** — plus what Accenture actually tests in its Online Assessment (OA) and Technical + HR interview. Follow it day-by-day. Every section ends with **what to say in the interview**.

Your Stack	Your Proof	Accenture Tests
Python, JS, SQL Scikit-learn, OpenCV, Tesseract, HF LLMs FastAPI, SQLModel, Postgres, Alembic, Docker, pytest React, Vite, Tailwind, Recharts	CampusPilot (OCR+LLM, 63+ tests) pocketF (Decimal CSV, burn-rate, Docker) UrTeam (Jaccard match, 15+ APIs) SIH SkyGuard, Patent, Jagrati	OA: Aptitude + Pseudocode + CS + 2 Coding Tech Interview: Resume deep-dive + CS fund. HR: Why Accenture, strengths, relocation

How to use: Days 1-2 = OA + CS fundamentals. Day 3 = Backend + Frontend depth. Day 4 = ML/AI + Projects storytelling. Day 5 = Mocks + HR. Each **Q/A box** is a real interview question you must answer aloud in 60-90 seconds.

Golden rule for Accenture: They hire *learnable, communicative engineers*. Always structure answers as: **What** → **How** → **Why this choice** → **Result/metric**. Quote numbers from your resume: 63+ tests, 15+ endpoints, 9 error types, 8 components, ~1500 students.

1. Accenture Hiring Pattern 2025-26 (What You Will Face)

1.1 Online Assessment (OA) — 90-120 min, online

Section A — Cognitive & Aptitude (40-50 Q): Quantitative (%, profit-loss, time-speed, series), Logical (blood relation, direction, coding-decoding, seating), Verbal (RC, synonyms, error spotting). Speed matters: ~1 min/Q.

Section B — Technical MCQs (30-40 Q): Pseudocode output (loops, bitwise, recursion), OOPs, DBMS/SQL, OS, CN basics, Cloud/SDLC/Agile. Plus 2-5 questions on Python/Java.

Section C — Coding (2 problems, 45-60 min): Typically 1 Easy (string/array/count) + 1 Medium (hashmap/two-pointer/simulation). Languages allowed: C/C++/Java/Python. Evaluation is on test cases + edge cases.

Section D — Communication (sometimes): Listening, sentence repeat, JAM (speak 1 min). Speak slowly, clearly.

1.2 Interview (1-2 rounds, 20-40 min)

Technical: 70% resume + projects, 20% CS fundamentals (OOPs, SQL joins, OS, API), 10% live code (reverse string, SQL query, API design). **HR:** Introduce yourself, why Accenture, strengths/weakness, relocation/shifts, bond/service agreement questions.

Round	They ask	You prepare (this guide)
OA Coding	Arrays, strings, dict, sorting	Sec 15 playbook — 15 patterns

Tech Interview	Explain CampusPilot OCR+LLM; Decimal vs float; Jaccard; FastAPI vs Flask; pytest/Docker	Sec 4-13 project Q&A;
HR	Why Accenture? Team conflict? Mechanical → IT?	Sec 17 STAR stories

2. Your 5-Day Stepwise Plan (Few Days Left)

Follow exactly — 6-8 hrs/day

Day	Tasks + Outcome
Day 1 — OA Aptitude + Pseudocode + Python	Aptitude formulas (Sec 16) 2h → Pseudocode drills 1.5h → Python depth (Sec 4) 2h → 2 easy codes (Sec 15) 1.5h. *Outcome: clear OA cut-off.*
Day 2 — CS Fundamentals + SQL + Coding	OOPs/OS/CN/DBMS (Sec 14) 3h → SQL joins + Postgres (Sec 5) 2h → 2 medium codes + dry-run 2h. *Outcome: answer any MCQ.*
Day 3 — Backend + Frontend Depth	FastAPI/REST/SQLModel/Alembic (Sec 6) 2.5h → React/Vite/Tailwind/Recharts (Sec 7) 2h → pytest/Docker/Git (Sec 9) 1.5h → speak UrTeam story aloud 1h.
Day 4 — ML/AI + Your 3 Projects	ML stack (Sec 8) 3h → CampusPilot + pocketF + SIH Q&A; (Sec 10-13) 3h → record yourself 60-sec per project. *Outcome: no resume question surprises you.*
Day 5 — Mocks + HR + Revision	Full OA mock 2h → Tech mock with friend 1.5h → HR STAR (Sec 17) 1.5h → 1-page revision sheet + early sleep.

Daily habit: 30 min loud speaking. Interview = verbal exam. If you can't say it, you don't know it.

3. Resume Master Map — Every Keyword, What They'll Ask

If a word is on your resume, you must defend it. Table below = your cheat sheet. Learn the **One-line defense** by heart.

3.1 Languages & Core

Q: Why Python as primary, not Java?
A: Python lets me ship ML + API fast with one language — Scikit-learn/Pandas for logic, FastAPI for serving. JavaScript only for UI. For Accenture enterprise work I can pick up Java quickly because OOP concepts transfer; Python already taught me OOP, exception handling, and virtualenv discipline.

Q: JavaScript — how deep?
A: ES6: let/const, arrow functions, promises/async-await, map/filter, destructuring. In React I use useState/useEffect, fetch/Axios to call FastAPI, conditional rendering. I know JS is single-threaded with event loop — so I never block UI with heavy compute.

Q: SQL — what level?
A: Joins (INNER/LEFT), GROUP BY + HAVING, subqueries, constraints (PK/FK/UNIQUE/NOT NULL), indexes, transactions (ACID). In Postgres I designed tables via SQLModel, used Alembic migrations, wrote analytics queries with Pandas read_sql for burn-rate reports.

3.2 ML/AI keywords

Q: Scikit-learn — what did you actually use?
A: Preprocessing (train_test_split, StandardScaler), models for tabular tasks, metrics (accuracy/precision/recall/F1). I know it is for classical ML on structured data — not deep learning. Deep learning would be PyTorch/TensorFlow.

Q: OpenCV — what for?
A: Image preprocessing before OCR: grayscale, thresholding, denoising, contour detection to crop seating-chart tables. I know cv2.imread returns NumPy array (H×W×C), and why you blur before threshold.

Q: Tesseract OCR — how does it work? Limitations?
A: OCR engine that segments characters and classifies via LSTM. Pipeline: preprocess with OpenCV → pytesseract.image_to_data → parse boxes → clean text → JSON. Fails on skewed/low-res scans and handwriting; I mitigated with DPI scaling and regex validation.

Q: Hugging Face + Phi-3 / Mistral / Gemma — what did you do?
A: Used HF transformers / inference API to convert noisy OCR text into structured JSON (roll → room → timetable). Chose small open models (Phi-3-mini, Mistral-7B, Gemma) for cost/latency. Used prompt templates + JSON-mode + validation, not fine-tuning. Know tokens, context window, temperature.

Q: Pandas / NumPy / Matplotlib — depth?
A: Pandas: read_csv, DataFrames, groupby, resampling for burn-rate; NumPy: vectorized ops, no Python loops; Matplotlib: auto-generated PNG charts for pocketF reports. Know the pitfall: float for money is wrong — hence Decimal.

3.3 Backend keywords

Q: FastAPI — why not Flask/Django?
A: FastAPI gives async, auto OpenAPI docs (/docs), and Pydantic validation out of box — perfect for ML APIs. Flask is minimal sync; Django is batteries-included monolith. I used layered architecture: routes → services → models, with dependency injection for DB sessions.

Q: REST APIs — what makes a good API?
A: Resource-based URLs (/students/{id}), correct verbs (GET/POST/PUT/DELETE), status codes (200/201/400/404/409/422), stateless, pagination/filtering (?tech=), versioning. I built 15+ endpoints with 409 duplicate detection and domain-validated emails.

Q: SQLModel + PostgreSQL + Alembic?
A: SQLModel = SQLAlchemy + Pydantic in one model class. Postgres = production relational DB with ACID, JSONB, concurrent writes. SQLite = local dev only. Alembic = migration tool (alembic revision --autogenerate + upgrade head) so schema changes don't wipe data.

Q: Docker + Docker Compose?
A: Docker packages app + deps into image via Dockerfile; Compose orchestrates api + postgres via docker-compose.yml (services, volumes, env). I containerized pocketF so examiner runs 'docker compose up' and gets API+DB. Know image vs container vs volume.

Q: pytest — 63+ tests means what?
A: Unit + API tests with pytest + FastAPI TestClient. I test happy path + edge cases (bad CSV rows, empty timetable, duplicate email → 409). 63 tests prove layered code is testable; CI can run 'pytest -q'. Know fixture, parametrize, assert.

3.4 Frontend keywords

Q: React / Vite / Tailwind / Recharts?
A: React: component model, props/state, hooks (useState/useEffect), SPA routing. Vite: fast dev server + build (vs CRA). Tailwind: utility-first CSS (I used inline utilities only, 8 reusable components). Recharts: declarative charts (LineChart/BarChart) for attendance/budget visuals. I know how React calls FastAPI via fetch and renders JSON.

4. Python Depth (Your Primary Language — Most Questions)

4.1 Must-know concepts with interview lines

OOP: class/object, __init__, inheritance, polymorphism (duck typing), encapsulation (_private, __name mangling), @classmethod/@staticmethod, @property. **Say:** 'I used OOP to model Student/Idea/Transaction as SQLModel classes with validation.'

Data structures: list vs tuple (mutable vs immutable), dict (hashmap O(1)), set (dedup + Jaccard), list comprehension, generators (yield, lazy). **Say:** 'For skill overlap I used sets — O(n) intersection vs O(n²) loops.'

Functions: *args/**kwargs, lambda, decorators (@app.get), closures. **Error handling:** try/except/else/finally, raise ValueError, custom exceptions for bad CSV rows.

Async & typing: async def + await for I/O-bound FastAPI routes; type hints (int|None, list[str]) + Pydantic for validation. **Memory:** GIL means threads don't parallelize CPU work — use multiprocessing or async I/O.

```
def jaccard(a: set[str], b: set[str]) -> float:
if not a and not b: return 0.0
return len(a & b) / len(a | b)
# Decimal for money — never float
from decimal import Decimal
total = sum((Decimal(r['amount']) for r in rows), Decimal('0'))
```

Q: List vs Tuple vs Dict vs Set — when to use?
A: List for ordered mutable, tuple for fixed records, dict for key lookup, set for uniqueness/intersection. Example: skills as sets for Jaccard; CSV rows as list of dicts; money totals as Decimal.

Q: What is decorator? Give example.
A: Function wrapping another to add behavior. I use @app.get('/x') and @pytest.fixture. Custom: @timer to log latency of OCR step.

Q: GIL?
A: Global Interpreter Lock — only one thread runs Python bytecode at once. Good for I/O with async/threads, bad for CPU loops. For CPU-heavy OCR I would use multiprocessing or offload to service.

5. SQL + PostgreSQL + SQLModel + Alembic

5.1 Core SQL you must write live

```
-- 1. Students with duplicate emails (your 409 logic)
SELECT email, COUNT(*) FROM student GROUP BY email HAVING COUNT(*)>1;
-- 2. Filter ideas by tech
SELECT * FROM idea WHERE tech ILIKE '%python%';
-- 3. Monthly spend (pocketF)
SELECT date_trunc('month', date) AS m, SUM(amount) FROM txn GROUP BY m ORDER BY m;
-- 4. LEFT JOIN: students even without team
SELECT s.name, t.name FROM student s LEFT JOIN team t ON t.id=s.team_id;
```

Joins in one line: INNER = only matches; LEFT = all left + matches; RIGHT = mirror; FULL = both; CROSS = cartesian. **Constraints:** PRIMARY KEY, FOREIGN KEY, UNIQUE (email), NOT NULL, CHECK (amount>0). **Index:** speeds WHERE/JOIN, slows writes. **ACID:** Atomicity, Consistency, Isolation, Durability — Postgres guarantees it; SQLite doesn't scale concurrent writes.

Q: Postgres vs SQLite?
A: SQLite is file-based, zero-config, single-writer — great for dev/demo. Postgres is server-based, handles concurrent writes, types (UUID, JSONB), and scales. My UrTeam auto-switches via DATABASE_URL.

Q: What is Alembic migration?
A: Versioned schema change. Workflow: edit SQLModel → alembic revision --autogenerate -m 'add team_id' → alembic upgrade head. Never delete DB in prod; migrate it.

Q: SQL injection? How did you prevent?
A: Never f-string SQL. Use ORM-bound params / SQLModel / SQLAlchemy placeholders. Plus Pydantic validation on inputs.

6. FastAPI + REST (Your Backend Spine)

6.1 Layered architecture (say this diagram aloud)

Client (React) → **Routes** (validate + status codes) → **Services** (business logic: Jaccard, burn-rate) → **Models** (SQLModel tables) → **Postgres**. OCR modules isolated so Tesseract/HF can be swapped. This layering is why you could write 63+ tests — services testable without HTTP.

```
from fastapi import FastAPI, HTTPException
app = FastAPI()
@app.get('/ideas')
def list_ideas(tech: str | None = None):
q = select(Idea)
if tech: q = q.where(Idea.tech.ilike(f'%{tech}%'))
return session.exec(q).all()
@app.post('/students', status_code=201)
def create(s: StudentCreate):
if exists(s.email): raise HTTPException(409, 'duplicate email')
...
```

Q: GET vs POST vs PUT vs PATCH vs DELETE?
A: GET read (idempotent), POST create, PUT full replace, PATCH partial update, DELETE remove. Idempotent = repeat safe (GET/PUT/DELETE).

Q: What happens when React calls your API? (End-to-end)
A: React fetch → DNS → TCP/TLS → HTTP GET → FastAPI route → Pydantic validation → service → SQLModel → Postgres → JSON response → React setState → re-render. Errors mapped to 400/404/409/422.

Q: How do you handle validation & errors?
A: Pydantic models (EmailStr, min_length) auto-return 422; business errors raise HTTPException(409/404) with detail; CSV importer collects per-row errors instead of failing whole file — 9 error types reported.

7. Frontend: React + Vite + Tailwind + Recharts

React mental model: UI = f(state). Component = function returning JSX. Props flow down, events flow up. useState for local data, useEffect for fetch on mount. Key={id} for lists. Conditional render {user && }.

```
// fetch FastAPI from React
useEffect(()=>{
fetch(`${import.meta.env.VITE_API}/ideas?tech=python`)
.then(r=>{ if(!r.ok) throw new Error(r.status); return r.json(); })
.then(setIdeas).catch(setError);
},[]);
```

Q: Vite vs CRA?
A: Vite uses native ESM + esbuild — instant start, fast HMR. CRA is webpack-based, slower. Both build to static files served by Nginx.

Q: Tailwind — why inline utilities?
A: No separate CSS files; classes like flex, gap-2, rounded-xl compose in JSX. Keeps 8 components consistent without CSS cascade bugs. Tradeoff: long classNames.

Q: Recharts?
A: Declarative React charts: . I fed it burn-rate JSON to show balance decline + SAFE/ALERT verdict.

8. ML / AI Stack — Speak Like a Builder, Not a Theorist

8.1 Scikit-learn + Pandas + NumPy + Matplotlib

Workflow: load (Pandas) → clean (fillna, encode) → split (train_test_split) → scale (StandardScaler) → model (LogReg/RandomForest/IsolationForest) → metric (accuracy/F1/RMSE) → plot (Matplotlib). **Overfitting:** train high, test low → regularize / cross-validate. **Say:** 'I use sklearn for tabular problems; LLMs for unstructured text.'

8.2 OpenCV + Tesseract (CampusPilot)

Pipeline you must recite in 60s: 1) PDF → images (pdf2image/poppler) 2) OpenCV: grayscale → GaussianBlur → adaptiveThreshold → findContours/deskew 3) pytesseract.image_to_data (PSM 6 for blocks) 4) regex clean roll numbers 5) HF LLM to JSON 6) validate + store. **PSM:** Page Segmentation Mode tells Tesseract layout. **DPI:** 300 DPI minimum for accuracy.

8.3 Hugging Face LLMs (Phi-3, Mistral, Gemma)

What they are: Open-weight transformers on HF Hub. Phi-3-mini (~3.8B, efficient), Mistral-7B (strong reasoning), Gemma (Google). **How you used:** prompt template: 'Extract JSON {roll, room, time} from: ...' + temperature 0 + JSON repair + schema validation. No fine-tuning — zero/few-shot. **Tokens:** subword units; context window = max input length; temperature 0 = deterministic.

Q: Why OCR + LLM, not OCR alone?
A: OCR gives noisy text; layout varies. LLM normalizes to schema, fixes splits ('12 34' → '1234'), maps synonyms. Validation layer rejects hallucinations — every LLM output checked against regex/DB.

Q: Isolation Forest + SHAP (SkyGuard)?
A: Isolation Forest isolates anomalies by random splits — anomalies need fewer splits; anomaly score = path length. For weather stations it flags sensor faults vs real extremes. SHAP explains 'why' via Shapley values — which sensor/feature pushed the decision. I proposed, not yet built — be honest.

9. Quality: pytest + Docker + Git (Accenture Loves This)

```
# pytest example you should explain
def test_duplicate_email_returns_409(client):
client.post('/students', json={'name':'A','email':'a@x.com'})
r = client.post('/students', json={'name':'B','email':'a@x.com'})
assert r.status_code == 409
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn","main:app","--host","0.0.0.0"]
```

Q: What do 63+ tests prove?
A: Layered design, edge-case thinking, regression safety. Unit tests for Jaccard/burn-rate math; API tests for CRUD + 409 + filters; importer tests for 9 CSV error types. Run with pytest -q + coverage.

Q: Docker image vs container vs volume vs compose?
A: Image = blueprint (read-only). Container = running instance. Volume = persistent data (postgres-data). Compose = YAML orchestrating api+db with networks/env. Commands: docker build -t api . / docker compose up --build.

Q: Git workflow?
A: main protected; feature branches; small commits; pull + PR review; .gitignore for .env. Commands: clone, branch, add/commit/push, pull --rebase, merge.

10. Project 1 — CampusPilot (Your Flagship, Expect 5-8 min)

One-minute pitch (memorize): 'CampusPilot is an academic platform for ~1500 students. Problem: exam seating and timetables are PDFs — students can't search roll numbers. I built an OCR+LLM pipeline with Tesseract and Phi-3/Mistral that converts seating PDFs to JSON, plus attendance tracking with Safe/Warning/Critical flags and a can-miss calculator, served via FastAPI + React with 63+ tests in a layered architecture. Next: React Native for mobile.'

Architecture to draw on paper

PDF → pdf2image → OpenCV preprocess → Tesseract → text clean → HF LLM → JSON validate → FastAPI (routes/services/models/ocr) → Postgres → React (lookup, timetable PDF via ReportLab, attendance charts). ReportLab generates personalized PDF timetables server-side.

Q: How do you compute Safe/Warning/Critical + can-miss?
A: attendance% = present/total. Thresholds e.g. >=75 Safe, 65-75 Warning, <65 Critical. Can-miss = floor((present - 0.75*total)/0.75) — max bunks while staying >=75%. Live current/next class by parsing timetable CSV/PDF + system time.

Q: What was hardest bug?
A: Skewed scans + merged cells broke Tesseract boxes. Fixed with deskew + DPI upscale + PSM tuning + regex post-pass; added per-page confidence log so low-confidence pages flagged for manual check instead of silent error.

Q: Why layered architecture?
A: Routes thin, services pure (testable), models typed. Lets me mock DB/OCR in tests — reason for 63 tests. New OCR engine = swap one module.

Q: ReportLab?
A: Server-side PDF lib. I build timetable PDFs with Platypus (Paragraph/Table) — same lib used for this guide. Handles fonts, page templates.

11. Project 2 — pocketF (Solo — Shows Ownership)

Pitch: 'pocketF is a student budgeting app. Users upload bank CSVs; I import with Decimal-exact math, report bad rows across 9 error types without aborting, compute burn-rate and money-runs-out date with SAFE/ALERT verdict, auto-generate Matplotlib charts, all containerized with Docker Compose and tested with pytest.'

```
from decimal import Decimal, InvalidOperation
errors=[]
for i,row in enumerate(rows,1):
try:
amt = Decimal(row['amount'].strip())
except (InvalidOperation, AttributeError):
errors.append({'row':i,'type':'BAD_AMOUNT'}); continue
```

Q: Why Decimal, not float?
A: Float is binary — 0.1+0.2 != 0.3. Money needs base-10 exact. Decimal('0.1')+Decimal('0.2')==Decimal('0.3'). I sum with Decimal starting at Decimal('0') and quantize to 2 places.

Q: What are the 9 CSV error types? (Be ready to list ~6)
A: Missing column, empty amount, bad decimal, bad date, future date, duplicate row, negative where not allowed, unknown category, encoding/BOM issue. Each reported as {row, type, message} so user fixes only bad rows.

Q: Burn-rate + runs-out forecast?
A: burn = avg daily spend over last N days (Pandas). remaining = balance; days_left = remaining/burn; end_date = today + days_left. If end_date > month_end → SAFE else ALERT. Edge cases: zero spend → Infinity guard; no data → 'insufficient data'. Tested with pytest.

Q: Docker Compose setup?
A: Two services: api (FastAPI, port 8000) + db (postgres:16, volume pgdata). Env via .env (DATABASE_URL). One

command: docker compose up --build. Shows you understand deployment, not just localhost.

12. Project 3 — UrTeam (Teamwork + Full-Stack)

Pitch: 'UrTeam matches students into hackathon teams by skill overlap using Jaccard similarity — no ML needed. I built 15+ REST endpoints with 409 duplicate handling and ?tech= filtering, dual SQLite/Postgres setup, owner-verified chat approvals, and 8 reusable React+Tailwind components.'

```
skills_a={'python','react'}; skills_b={'python','ml'}
j = len(skills_a & skills_b)/len(skills_a | skills_b) # 0.33
```

Q: Why Jaccard, not ML?
A: Skills are sets, not vectors. Jaccard = |A∩B|/|A∪B| is interpretable, O(n), zero deps. ML (embeddings) overkill for small data and harder to explain in interview. I can upgrade to TF-IDF/cosine later.

Q: Explain 409 duplicate + ?tech= filter?
A: On POST /students, check email exists → 409 Conflict (not 400) so client knows it's duplicate, not bad syntax. GET /ideas?tech=python filters via ILIKE; keeps API RESTful instead of /getPythonIdeas.

Q: Dual DB + auto table creation?
A: SQLModel.metadata.create_all for dev velocity; DATABASE_URL switches SQLite (local) ↔ Postgres (prod). For prod you'd use Alembic migrations instead of create_all to avoid data loss.

Q: Chat approval + owner-verified access?
A: Membership request → pending → owner approves. Every mutating route checks requester == owner_id else 403 Forbidden. Prevents anyone joining/reading private chats. Auth simplified for demo; prod needs JWT.

13. SIH + Patent + Jagrati (Differentiators)

Q: SkyGuard AI — explain in 90s (SIH26073)?
A: Problem: Automatic Weather Stations produce faults that look like extreme weather. Proposed Isolation Forest to score anomalies on temp/humidity/wind streams, plus SHAP to explain which sensor caused flag, so IMD staff trust alerts. Team of 6; my role: ML pipeline + API sketch. Status: proposed/prototype — be honest you haven't deployed at scale.

Q: Patent 202621096766 A — what to say as mechanical student?
A: Co-inventor of adjustable hybrid welding device with integrated safety. Explain problem (fixed jigs don't fit varied jobs + safety gaps), your contribution (adjustable mechanism / safety interlock — keep to your actual part), status: application published, not granted. Shows cross-disciplinary engineering + documentation rigor. Don't overclaim — say 'application no. ... A'.

Q: Jagrati volunteering?
A: Teach village kids evenings + plan activities. Accenture values community. Frame with STAR: led 5 kids in math, improved attendance by X, learned patience/communication — same skills for client teams.

Q: Mechanical → Software? (They WILL ask)
A: Mechanical taught me systems thinking, CAD tolerances, thermodynamics problem-solving. I applied same rigor to software: layered design, testing, Docker reproducibility. Patent + SIH show I bridge both. I want Accenture because enterprise scale lets me apply engineering + AI together.

14. Core CS for Accenture MCQs + Interview

OOPs (4 pillars, 1 line each)

Encapsulation (bundle data+methods, private), Inheritance (reuse via parent), Polymorphism (same interface, different behavior), Abstraction (hide complexity via API). Plus overloading (same name diff args — Python via defaults) vs overriding (child redefines).

OS essentials

Process vs thread (isolated vs shared memory); concurrency vs parallelism; deadlock (4 Coffman conditions); paging/virtual memory; mutex/semaphore; `ps/top/kill`, zombie vs orphan. GIL ties back to Python threads.

CN essentials

OSI 7 vs TCP/IP 4; TCP (reliable, handshake) vs UDP (fast); HTTP methods/status (200/201/400/401/403/404/409/422/500); DNS → IP; 3-way handshake SYN/SYN-ACK/ACK; JWT = header.payload.signature for auth.

SDLC + Agile + Cloud (Accenture favorites)

SDLC: Req→Design→Build→Test→Deploy→Maintain. Agile: sprints, standup, backlog, retro — vs Waterfall. Cloud: IaaS/PaaS/SaaS; Docker = container, K8s = orchestration; CI/CD = test+deploy on push. Say: 'I practice CI mindset with pytest + Docker.'

15. OA Coding Playbook — 15 Patterns, Python First

Accenture coding = input parsing + edge cases + O(n) thinking. Master these 5 first: frequency map, two pointers, sliding window, sorting+greedy, hashset. Always: read n, strip, handle empty, print exactly as asked.

```
# template that never fails
import sys
def solve():
data = sys.stdin.read().strip().split()
if not data: return
n = int(data[0]); arr = list(map(int, data[1:1+n]))
print(sum(arr))
if __name__=='__main__': solve()
```

```
# 1 frequency 2 two-sum 3 Jaccard (your project!)
from collections import Counter
def freq(s): return Counter(s)
def two_sum(a,t):
seen={}
for i,x in enumerate(a):
if t-x in seen: return (seen[t-x],i)
seen[x]=i
def jaccard(a,b): return len(set(a)&set(b))/len(set(a)|set(b)) if (a or b) else 0.0
```

```
# 4 reverse words 5 valid parentheses 6 burn-rate forecast
def rev_words(s): return ' '.join(s.split()[::-1])
def valid(p):
st=[]; m={')':'(',']':'[','}':'{'}
for c in p:
if c in '([{': st.append(c)
elif not st or st.pop()!=m[c]: return False
return not st
```

Practice list (do 2/day): reverse string, palindrome, anagram, second largest, missing number, move zeros, leaders in array, balanced parens, first non-repeating char, Jaccard/top-k match, CSV bad-row counter, monthly aggregate, merge intervals, stock buy-sell once, queue via two stacks. Time yourself 25 min each; test with empty/single/duplicate/large inputs.

Q: How to explain code in interview?
A: State approach + complexity first: 'Hashmap O(n) time O(n) space because...', then walk dry-run, then edge cases. Never stay silent — think aloud.

16. Aptitude + Pseudocode Cheat Sheet (OA Survival)

Quant formulas: % = (part/whole)*100; SP=CP*(1±%/100); SI=PRT/100; Speed=Dist/Time; upstream/downstream avg = 2xy/(x+y); series: check diff, ratio, squares (1,4,9,16), primes. **Logic:** draw tables for seating; Venn for sets; assume + test options backwards when stuck.

Pseudocode traps: integer division (5/2=2 in C), modulo with negatives, pre vs post ++ (x++ returns then increments), && short-circuit, pass-by-value vs reference, recursion base case, bitwise (<< = *2, >> = /2, ^ = XOR). Always dry-run tiny input (n=2) on paper.

```
// example: what prints?
// for(i=0;i<3;i++) for(j=0;j<=i;j++) print('*') => 1+2+3=6 stars
# python trap: [0]*3 + list aliasing; range(5)[::-1]
```

17. HR + Behavioral — STAR Answers That Win

Introduce yourself (60s, memorize): 'I'm Rishikesh, Mechanical undergrad at IIITDM Jabalpur. I build ML-powered apps end-to-end — OCR+LLM timetables, budgeting forecasts, team matchmaking — with FastAPI, Postgres, React, Docker and 60+ tests. I co-invented a welding-device patent, proposed SkyGuard AI for SIH, and teach with Jagrati. I'm seeking an AI/ML or SDE internship at Accenture to learn enterprise delivery.'

Q: Why Accenture?
A: Scale + learning: global clients, structured training, Agile delivery, and AI/cloud practice match my stack (Python/APIs/ML). I want mentorship + real production discipline — Docker/testing habits I started solo, I want to practice at enterprise scale. Plus values: innovation + community (Jagrati resonates).

Q: Strengths / Weakness?
A: Strength: end-to-end ownership — solo pocketF from CSV math to Docker. Weakness: I over-engineer; now I timebox MVP then add tests. Show growth.

Q: Team conflict? (STAR)
A: Situation: SIH team split on model choice. Task: agree in 1 day. Action: I proposed 2-hr spike: Isolation Forest vs threshold, compared precision on sample faults. Result: team aligned on IF+SHAP, doc'd decision. Learning: data beats debate.

Q: Relocation / shifts / bond?
A: Yes, flexible for relocation and shifts; understand Accenture delivery needs. (Only agree if true; keep passport/docs ready.)

Q: Where in 2 years?
A: Strong SDE/ML engineer owning microservices + ML pipelines with testing/DevOps discipline, mentoring juniors — grown through Accenture learning paths.

18. Final Checklists

OA day checklist

ID + laptop charger + stable net; rough sheets; read Q twice; solve easy code first for confidence; handle empty input; print format exactly; reserve 5 min to re-run samples. If stuck >5 min, switch section — cut-off needs attempts.

Interview day checklist

Open GitHub repos + live demos; draw architecture on paper; speak numbers (63 tests, 15 endpoints, 9 errors); admit what you didn't do (no fine-tuning, prototype SIH); ask 1 smart Q: 'How does Accenture review junior code / what testing bar for ML APIs?'

One-page revision (night before)

Python: GIL, decorator, Decimal. SQL: joins, HAVING, ACID. API: 200/201/409/422, layered arch. React: hooks, fetch. ML: OCR pipeline, temp=0 JSON, Jaccard formula, IF+SHAP one-liner. Projects: 60-sec pitch each. HR: intro + why Accenture + STAR conflict.

You are ready when: you can whiteboard CampusPilot pipeline, write Jaccard + Decimal + join query without googling, and speak each project for 60s without 'umm'. Good luck — you built real things; now narrate them confidently.